

HDOC DAY-20 ASSIGNMENT

Application of Iterative and Decision-Making Constructs to Shapes and Physics using Modular Programming in C

1. Problem Understanding

The task is to develop a menu-driven modular C project that performs geometrical and physical calculations for a sphere, cylinder, cone, and cuboid. The project uses fundamental data types, functions with call by value, selection statements, and loops. It does not use arrays, pointers, recursion, structures, unions, typedef, or functions from math.h.

- Calculate total surface area, volume, and base area of supported solids.
- Calculate mass, weight, gravitational potential energy, pressure, and buoyant force.
- Determine whether a completely immersed solid sinks, floats, or remains suspended.
- Calculate square root iteratively using the Newton-Raphson method for the cone slant height.
- Validate menu choices, dimensions, densities, and elevation before calculation.

2. Standard Test Cases

Test Case	Choice	Complete Input	Expected Output	Type
TC1	1 - Surface Area	Sphere; r = 2 m	50.2656 m ²	Valid
TC2	2 - Volume	Cylinder; r = 2 m, h = 5 m	62.8320 m ³	Valid
TC3	3 - Base Area	Cone; r = 3 m	28.2744 m ²	Valid
TC4	4 - Mass	Cuboid; l=2 m, b=3 m, h=4 m; density=500 kg/m ³	12000.0000 kg	Valid
TC5	5 - Weight	Sphere; r=1 m; density=1000 kg/m ³	41050.2400 N	Valid
TC6	6 - Potential Energy	Cylinder; r=1 m, h=2 m; density=500 kg/m ³ ; elevation=10 m	307876.8000 J	Valid
TC7	7 - Pressure	Cuboid; l=2 m, b=3 m, h=4 m; density=500 kg/m ³	19600.0000 Pa	Valid
TC8	8 - Buoyant Force	Sphere; r=1 m; fluid density=1000 kg/m ³	41050.2400 N	Valid
TC9	9 - Buoyancy	solid density=800; fluid density=1000 kg/m ³	Solid floats	Valid
TC10	9 - Buoyancy	solid density=1000; fluid density=1000 kg/m ³	Solid remains suspended	Boundary

3. Algorithm

A. Common Menu Logic

1. Start.
2. Display the main menu with options 1 to 10.
3. Read the user choice.
4. Validate the choice. If it is outside 1 to 10, display an error message.
5. Use switch-case to call the required operation.
6. For geometry operations, ask the user to select the solid and enter the required dimensions.
7. Validate every dimension and density so that it is greater than zero. Elevation must not be negative.
8. Display the calculated result in SI units.
9. Repeat the menu using a do-while loop until choice 10 (Exit) is selected.
10. Stop.

B. Surface Area

11. Read the solid choice.
12. Read and validate the required dimensions.

13. For sphere, return $4 * \text{PI} * \text{radius} * \text{radius}$.
14. For cylinder, return $2 * \text{PI} * \text{radius} * (\text{radius} + \text{height})$.
15. For cone, calculate slant height using `calculate_square_root(radius*radius + height*height)`, then return $\text{PI} * \text{radius} * (\text{radius} + \text{slant_height})$.
16. For cuboid, return $2 * (\text{length} * \text{breadth} + \text{breadth} * \text{height} + \text{length} * \text{height})$.
17. Display area in square metres.

C. Volume

18. Read the solid choice and required positive dimensions.
19. Sphere: return $(4.0/3.0) * \text{PI} * \text{radius} * \text{radius} * \text{radius}$.
20. Cylinder: return $\text{PI} * \text{radius} * \text{radius} * \text{height}$.
21. Cone: return $(1.0/3.0) * \text{PI} * \text{radius} * \text{radius} * \text{height}$.
22. Cuboid: return $\text{length} * \text{breadth} * \text{height}$.
23. Display volume in cubic metres.

D. Base Area

24. Read the solid choice.
25. If sphere is selected, display that base area is not defined and return to the main menu.
26. For cylinder or cone, return $\text{PI} * \text{radius} * \text{radius}$.
27. For cuboid, return $\text{length} * \text{breadth}$.
28. Display base area in square metres.

E. Mass

29. Read density and validate it as greater than zero.
30. Read solid type and dimensions.
31. Calculate volume using the appropriate geometry function.
32. Calculate mass = density * volume.
33. Return and display mass in kilograms.

F. Weight

34. Obtain mass by calling `calculate_mass()`.
35. Calculate weight = mass * GRAVITY.
36. Return and display weight in newtons.

G. Gravitational Potential Energy

37. Read elevation and validate that it is not negative.
38. Obtain mass by calling `calculate_mass()`.
39. Calculate potential energy = mass * GRAVITY * elevation.
40. Return and display energy in joules.

H. Pressure on the Base

41. Read density and validate it.
42. Read solid type. Permit only cylinder, cone, and cuboid.
43. Calculate volume using the appropriate volume function.
44. Calculate contact area using `circular_base_area()` or `cuboid_base_area()`.
45. Calculate pressure = (density * volume * GRAVITY) / contact_area.
46. Return and display pressure in pascals.

I. Buoyant Force

47. Read fluid density and validate it.
48. Read solid type and dimensions.
49. Calculate volume using the appropriate volume function.
50. Calculate buoyant force = fluid_density * volume * GRAVITY.

51. Return and display force in newtons.

J. Buoyancy Behavior

- 52. Read solid density and fluid density and validate both.
- 53. If solid_density > fluid_density, return SINKS.
- 54. If solid_density < fluid_density, return FLOATS.
- 55. Otherwise return SUSPENDED.
- 56. Display the corresponding message.

K. Newton-Raphson Square Root

- 57. If number is negative, return -1.
- 58. If number is zero, return 0.
- 59. Set old_value = number.
- 60. Calculate new_value = $0.5 * (old_value + number / old_value)$.
- 61. Find difference = new_value - old_value and make it positive if negative.
- 62. Set old_value = new_value.
- 63. Repeat while difference >= TOLERANCE.
- 64. Return new_value.

4. C Program

The project is divided into five files as required: solid.h, geometry.c, physics.c, math_utils.c, and main.c. The program is menu-driven and uses simple functions, switch-case, loops, and basic variables.

A. solid.h

```
#ifndef SOLID_H
#define SOLID_H

#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);
double calculate_mass(double density);
double calculate_weight();
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);
double calculate_square_root(double number);

#endif
```

B. geometry.c

```
#include "solid.h"

double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}

double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}

double cone_area(double radius, double height)
{
    double slant_height;
    slant_height = calculate_square_root(radius * radius + height * height);
    return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height)
{
    return (1.0 / 3.0) * PI * radius * radius * height;
}

double cuboid_area(double length, double breadth, double height)
{
    return 2 * (length * breadth + breadth * height + length * height);
}

double cuboid_volume(double length, double breadth, double height)
{
    return length * breadth * height;
}

double circular_base_area(double radius)
{
    return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth)
{
    return length * breadth;
}
```

C. math_utils.c

```
#include "solid.h"

double calculate_square_root(double number)
{
    double old_value, new_value, difference;
```

```

if (number < 0)
    return -1;
if (number == 0)
    return 0;

old_value = number;
do
{
    new_value = 0.5 * (old_value + number / old_value);
    difference = new_value - old_value;
    if (difference < 0)
        difference = -difference;
    old_value = new_value;
} while (difference >= TOLERANCE);

return new_value;
}

```

D. physics.c

```

#include <stdio.h>
#include "solid.h"

double calculate_mass(double density)
{
    int solid;
    double r, h, l, b, volume = 0;

    printf("1. Sphere 2. Cylinder 3. Cone 4. Cuboid\n");
    printf("Enter solid: ");
    scanf("%d", &solid);
    while (solid < 1 || solid > 4)
    {
        printf("Invalid solid. Enter again: ");
        scanf("%d", &solid);
    }

    if (solid == SPHERE)
    {
        printf("Enter radius: "); scanf("%lf", &r);
        while (r <= 0) { printf("Enter positive radius: "); scanf("%lf", &r); }
        volume = sphere_volume(r);
    }
    else if (solid == CYLINDER || solid == CONE)
    {
        printf("Enter radius and height: "); scanf("%lf %lf", &r, &h);
        while (r <= 0 || h <= 0)
        { printf("Enter positive radius and height: "); scanf("%lf %lf", &r, &h); }
        if (solid == CYLINDER) volume = cylinder_volume(r, h);
        else volume = cone_volume(r, h);
    }
    else
    {
        printf("Enter length, breadth and height: "); scanf("%lf %lf %lf", &l, &b, &h);
        while (l <= 0 || b <= 0 || h <= 0)
        { printf("Enter positive values: "); scanf("%lf %lf %lf", &l, &b, &h); }
        volume = cuboid_volume(l, b, h);
    }
    return density * volume;
}

double calculate_weight()
{
    double density, mass;
    printf("Enter density: "); scanf("%lf", &density);

```

```

while (density <= 0) { printf("Enter positive density: "); scanf("%lf", &density); }
mass = calculate_mass(density);
return mass * GRAVITY;
}

```

```

double potential_energy(double elevation)
{
    double density, mass;
    printf("Enter density: "); scanf("%lf", &density);
    while (density <= 0) { printf("Enter positive density: "); scanf("%lf", &density); }
    mass = calculate_mass(density);
    return mass * GRAVITY * elevation;
}

```

```

double calculate_pressure(double density)
{
    int solid;
    double r, h, l, b, volume, area;
    printf("2. Cylinder 3. Cone 4. Cuboid\nEnter solid: ");
    scanf("%d", &solid);
    while (solid < 2 || solid > 4)
    { printf("Invalid solid. Enter 2, 3 or 4: "); scanf("%d", &solid); }

    if (solid == CYLINDER || solid == CONE)
    {
        printf("Enter radius and height: "); scanf("%lf %lf", &r, &h);
        while (r <= 0 || h <= 0)
        { printf("Enter positive radius and height: "); scanf("%lf %lf", &r, &h); }
        area = circular_base_area(r);
        if (solid == CYLINDER) volume = cylinder_volume(r, h);
        else volume = cone_volume(r, h);
    }
    else
    {
        printf("Enter length, breadth and height: "); scanf("%lf %lf %lf", &l, &b, &h);
        while (l <= 0 || b <= 0 || h <= 0)
        { printf("Enter positive values: "); scanf("%lf %lf %lf", &l, &b, &h); }
        area = cuboid_base_area(l, b);
        volume = cuboid_volume(l, b, h);
    }
    return (density * volume * GRAVITY) / area;
}

```

```

double buoyant_force(double fluid_density)
{
    int solid;
    double r, h, l, b, volume = 0;
    printf("1. Sphere 2. Cylinder 3. Cone 4. Cuboid\nEnter solid: ");
    scanf("%d", &solid);
    while (solid < 1 || solid > 4)
    { printf("Invalid solid. Enter again: "); scanf("%d", &solid); }

    if (solid == SPHERE)
    { printf("Enter radius: "); scanf("%lf", &r); while (r <= 0) { printf("Enter positive radius: "); scanf("%lf", &r); } volume = sphere_volume(r); }
    else if (solid == CYLINDER || solid == CONE)
    { printf("Enter radius and height: "); scanf("%lf %lf", &r, &h); while (r <= 0 || h <= 0) { printf("Enter positive values: "); scanf("%lf %lf", &r, &h); } if
(solid == CYLINDER) volume = cylinder_volume(r,h); else volume = cone_volume(r,h); }
    else
    { printf("Enter length, breadth and height: "); scanf("%lf %lf %lf", &l,&b,&h); while (l <= 0 || b <= 0 || h <= 0) { printf("Enter positive values: ");
scanf("%lf %lf %lf", &l,&b,&h); } volume = cuboid_volume(l,b,h); }
    return fluid_density * volume * GRAVITY;
}

```

```

int determine_buoyancy(double solid_density, double fluid_density)
{
    if (solid_density > fluid_density) return SINKS;
    else if (solid_density < fluid_density) return FLOATS;
    else return SUSPENDED;
}

```

E. main.c

```

#include <stdio.h>
#include "solid.h"

int main()
{
    int choice, solid, result;
    double r, h, l, b, density, fluid_density, solid_density, elevation;

    do
    {
        printf("\n----- MAIN MENU ----- \n");
        printf("1. Surface Area\n2. Volume\n3. Base Area\n4. Mass\n5. Weight\n");
        printf("6. Gravitational Potential Energy\n7. Pressure on Base\n");
        printf("8. Buoyant Force\n9. Buoyancy Behaviour\n10. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
            case 2:
            case 3:
                printf("1. Sphere 2. Cylinder 3. Cone 4. Cuboid\nEnter solid: ");
                scanf("%d", &solid);
                while (solid < 1 || solid > 4)
                { printf("Invalid solid. Enter again: "); scanf("%d", &solid); }

                if (choice == 3 && solid == SPHERE)
                { printf("Base area of sphere is not defined.\n"); break; }

                if (solid == SPHERE)
                {
                    printf("Enter radius: "); scanf("%lf", &r);
                    while (r <= 0) { printf("Enter positive radius: "); scanf("%lf", &r); }
                    if (choice == 1) printf("Area = %.2lf sq m\n", sphere_area(r));
                    else printf("Volume = %.2lf cu m\n", sphere_volume(r));
                }
                else if (solid == CYLINDER || solid == CONE)
                {
                    printf("Enter radius and height: "); scanf("%lf %lf", &r, &h);
                    while (r <= 0 || h <= 0)
                    { printf("Enter positive radius and height: "); scanf("%lf %lf", &r, &h); }
                    if (choice == 1 && solid == CYLINDER) printf("Area = %.2lf sq m\n", cylinder_area(r,h));
                    else if (choice == 1) printf("Area = %.2lf sq m\n", cone_area(r,h));
                    else if (choice == 2 && solid == CYLINDER) printf("Volume = %.2lf cu m\n", cylinder_volume(r,h));
                    else if (choice == 2) printf("Volume = %.2lf cu m\n", cone_volume(r,h));
                    else printf("Base Area = %.2lf sq m\n", circular_base_area(r));
                }
            }
            else
            {
                printf("Enter length, breadth and height: "); scanf("%lf %lf %lf", &l,&b,&h);
                while (l <= 0 || b <= 0 || h <= 0)
                { printf("Enter positive values: "); scanf("%lf %lf %lf", &l,&b,&h); }
                if (choice == 1) printf("Area = %.2lf sq m\n", cuboid_area(l,b,h));
                else if (choice == 2) printf("Volume = %.2lf cu m\n", cuboid_volume(l,b,h));
                else printf("Base Area = %.2lf sq m\n", cuboid_base_area(l,b));
            }
        }
    }
}

```

```

    }
    break;

case 4:
    printf("Enter density: "); scanf("%lf", &density);
    while (density <= 0) { printf("Enter positive density: "); scanf("%lf", &density); }
    printf("Mass = %.2lf kg\n", calculate_mass(density));
    break;
case 5:
    printf("Weight = %.2lf N\n", calculate_weight());
    break;
case 6:
    printf("Enter elevation: "); scanf("%lf", &elevation);
    while (elevation < 0) { printf("Elevation cannot be negative. Enter again: "); scanf("%lf", &elevation); }
    printf("Potential Energy = %.2lf J\n", potential_energy(elevation));
    break;
case 7:
    printf("Enter density: "); scanf("%lf", &density);
    while (density <= 0) { printf("Enter positive density: "); scanf("%lf", &density); }
    printf("Pressure = %.2lf Pa\n", calculate_pressure(density));
    break;
case 8:
    printf("Enter fluid density: "); scanf("%lf", &fluid_density);
    while (fluid_density <= 0) { printf("Enter positive density: "); scanf("%lf", &fluid_density); }
    printf("Buoyant Force = %.2lf N\n", buoyant_force(fluid_density));
    break;
case 9:
    printf("Enter solid density: "); scanf("%lf", &solid_density);
    while (solid_density <= 0) { printf("Enter positive density: "); scanf("%lf", &solid_density); }
    printf("Enter fluid density: "); scanf("%lf", &fluid_density);
    while (fluid_density <= 0) { printf("Enter positive density: "); scanf("%lf", &fluid_density); }
    result = determine_buoyancy(solid_density, fluid_density);
    if (result == SINKS) printf("Solid sinks.\n");./
    else if (result == FLOATS) printf("Solid floats.\n");
    else printf("Solid remains suspended.\n");
    break;
case 10:
    printf("Program ended.\n");
    break;
default:
    printf("Invalid menu choice.\n");
}
} while (choice != 10);

return 0;
}

```

5. Compilation and Execution Evidence

```
kali@kali: ~/Desktop/day20

Session Actions Edit View Help

(kali@kali)~-[~/Desktop]
$ cd day20

(kali@kali)~-[~/Desktop/day20]
$ ./day20

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 1

Select Solid
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 1
Enter radius: 2
Surface Area = 50.27 square metres

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 2
```

```
kali@kali: ~/Desktop/day20

Session Actions Edit View Help

Select Solid
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 2
Enter radius: 2
Enter height: 5
Volume = 62.83 cubic metres

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 3

Select Solid
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 3
Enter radius: 3
Base Area = 28.27 square metres

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
```

```
kali@kali: ~/Desktop/day20

Session Actions Edit View Help

7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 4
Enter density: 500

Select Solid
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 4
Enter length: 2
Enter breadth: 3
Enter height: 4
Mass = 12000.00 kg

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 5
Enter density: 1000

Select Solid
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 1
Enter radius: 1
Weight = 41050.24 N
```

```
kali@kali: ~/Desktop/day20

Session Actions Edit View Help

Weight = 41050.24 N

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 6
Enter elevation: 10
Enter density: 500

Select Solid
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 2
Enter radius: 1
Enter height: 2
Potential Energy = 307876.80 J

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 7
Enter density: 500
```

```
kali@kali: ~/Desktop/day20
Session Actions Edit View Help
Enter density: 500

Select Solid
1. Cylinder
2. Cone
3. Cuboid
Enter choice: 3
Enter length: 2
Enter breadth: 3
Enter height: 4
Pressure = 19600.00 Pa

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 8
Enter fluid density: 1000

Select Solid
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 1
Enter radius: 1
Buoyant Force = 41050.24 N

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
```

```
kali@kali: ~/Desktop/day20
Session Actions Edit View Help
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density: 800
Enter fluid density: 1000
The solid floats.

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 9
Enter solid density: 1000
Enter fluid density: 1000
The solid remains suspended.

===== MAIN MENU =====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force
9. Buoyancy Behaviour
10. Exit
Enter your choice: 10
Program ended.

(kali@kali)-[~/Desktop/day20]
$
```

6. Verification of All Test Cases

Test Case	Choice	Input	Expected Result	Actual Result	Status
TC1	1 - Surface Area	Sphere; $r = 2 \text{ m}$	50.2656 m^2	50.2656 m^2	Pass
TC2	2 - Volume	Cylinder; $r = 2 \text{ m}$, $h = 5 \text{ m}$	62.8320 m^3	62.8320 m^3	Pass
TC3	3 - Base Area	Cone; $r = 3 \text{ m}$	28.2744 m^2	28.2744 m^2	Pass
TC4	4 - Mass	Cuboid; $l=2 \text{ m}$, $b=3 \text{ m}$, $h=4 \text{ m}$; density= 500 kg/m^3	12000.0000 kg	12000.0000 kg	Pass
TC5	5 - Weight	Sphere; $r=1 \text{ m}$; density= 1000 kg/m^3	41050.2400 N	41050.2400 N	Pass
TC6	6 - Potential Energy	Cylinder; $r=1 \text{ m}$, $h=2 \text{ m}$; density= 500 kg/m^3 ; elevation= 10 m	307876.8000 J	307876.8000 J	Pass
TC7	7 - Pressure	Cuboid; $l=2 \text{ m}$, $b=3 \text{ m}$, $h=4 \text{ m}$; density= 500 kg/m^3	19600.0000 Pa	19600.0000 Pa	Pass
TC8	8 - Buoyant Force	Sphere; $r=1 \text{ m}$; fluid density= 1000 kg/m^3	41050.2400 N	41050.2400 N	Pass
TC9	9 - Buoyancy	solid density= 800 ; fluid density= 1000 kg/m^3	Solid floats	Solid floats	Pass
TC10	9 - Buoyancy	solid density= 1000 ; fluid density= 1000 kg/m^3	Solid remains suspended	Solid remains suspended	Pass
